

Face Following

Face Following

1. Course Content
2. Start Agent
3. Run Case
 - 3.1 Dynamic Parameter Adjustment
4. Source Code Analysis

1. Course Content

- Run example programs, recognize detected faces and frame them, then the car will continuously follow the face to keep it in the center of the screen, and maintain a distance of 0.4 meters from the recognized face. Press the q/Q key to exit the program.

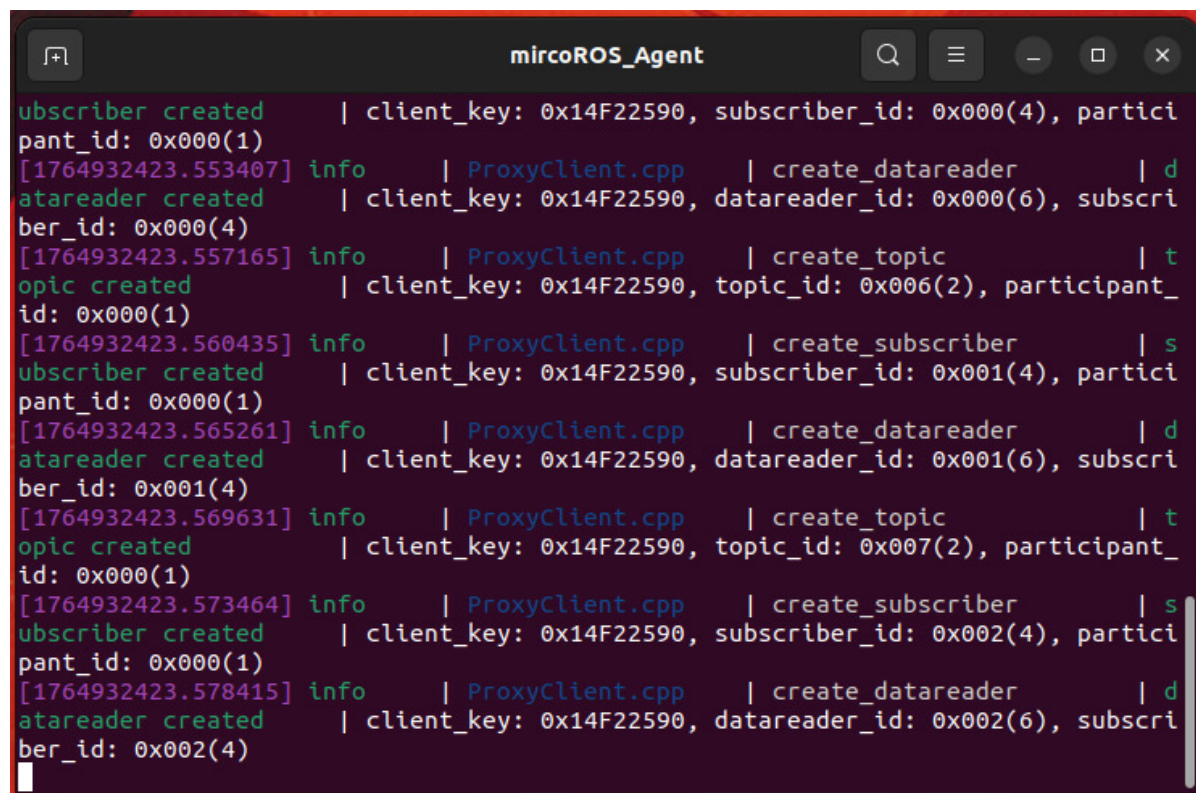
2. Start Agent

Note: If already started, no need to restart

Enter command in the car terminal:

```
start_agent
```

The terminal prints the following information, indicating successful connection



```
mircoROS_Agent
[1764932423.553407] info | ProxyClient.cpp | create_datareader | d
subscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1764932423.557165] info | ProxyClient.cpp | create_topic | t
topic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1764932423.560435] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1764932423.565261] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1764932423.569631] info | ProxyClient.cpp | create_topic | t
topic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1764932423.573464] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1764932423.578415] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

3. Run Case

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

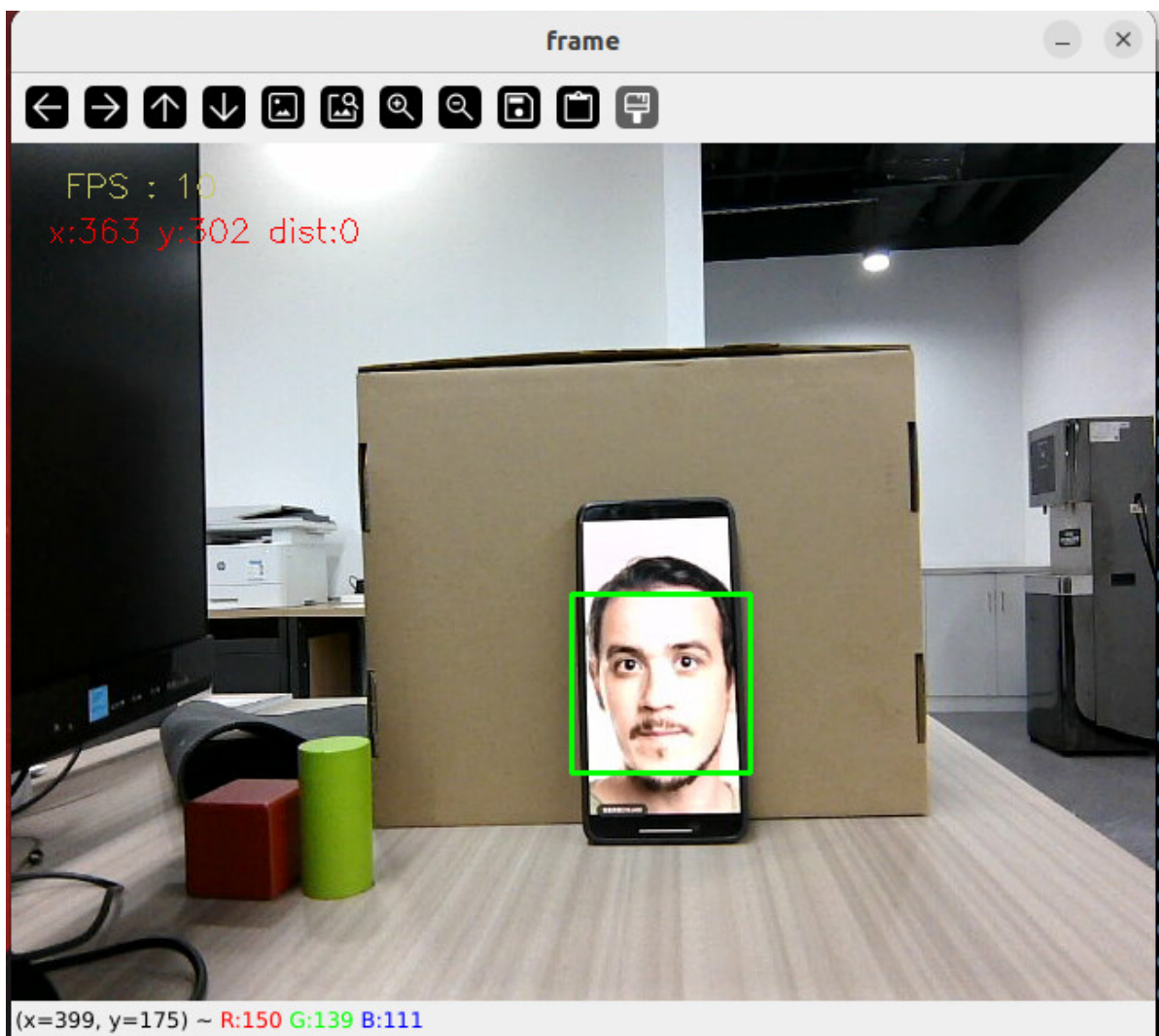
- Enter command in terminal:

```
ros2 launch m3_bringup depthcam_demo.launch.py demo:=FaceFollow
```

- When a face is detected, it will automatically frame the face area, and the car will start following.

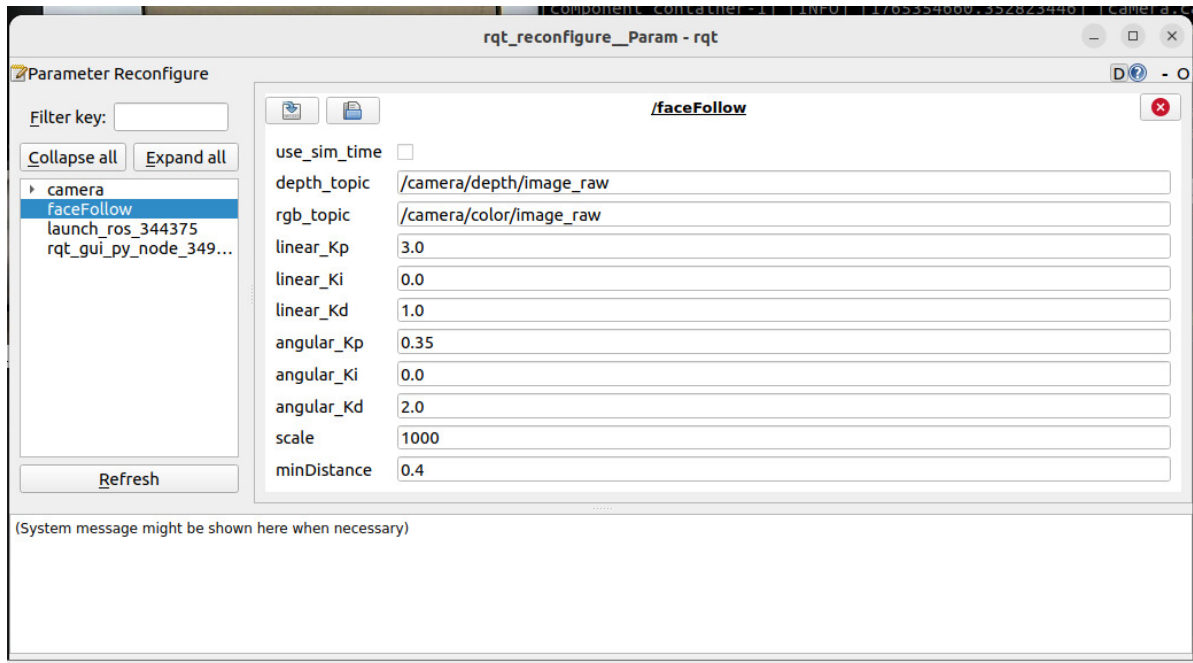
[!NOTE]

- If the car runs with poor stuttering effect, you can replace it with a face photo that has better depth information recognition. This is because if the face depth information is 0, the car will stop



3.1 Dynamic Parameter Adjustment

```
ros2 run rqt_reconfigure rqt_reconfigure
```



- After modifying parameters, click on the blank area of the GUI to write parameter values. Note that it will only take effect for the current startup. For permanent effect, you need to modify the parameters in the source code.

Parameter analysis:

【linear_Kp】 , 【linear_Ki】 , 【linear_Kd】 : Linear speed PID control during car following process.

【angular_Kp】 , 【angular_Ki】 , 【angular_Kd】 : Angular speed PID control during car following process.

【minDistance】 : Following distance, always maintain this distance.

4. Source Code Analysis

faceFollow.py source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/depthcamera/faceFollow.py
```

- This program mainly has the following functions:
 - Subscribe to depth camera topics to get topic images;
 - Use OpenCV's face detector to detect faces;
 - Process images to get face image center coordinates and depth distance information.
 - Calculate distance PID and center coordinate PID to get car forward speed distance and turning angle.

Some core code is as follows,

```
#Get the size of the box where the face is detected
```

```

faces = self.face_patterns.detectMultiScale(result_image , scaleFactor=1.2,
minNeighbors=10, minSize=(50, 50))
#Single face processing logic
if len(faces) == 1:
    #Face center calculation
    for (x, y, w, h) in faces:
        self.Center_x = x + w // 2 # X center point
        self.Center_y = y + h // 2 # Y center point
        self.Center_r = w * h // 100 # Area radius
        cv.rectangle(result_image, (x, y), (x+w, y+h), (0,255,0), 2) # Draw a
green frame
    #Deep information acquisition
    points = [
        (int(self.Center_y), int(self.Center_x)), # Center Point
        (int(self.Center_y-1), int(self.Center_x-1)), # Lower right offset point
        (int(self.Center_y-3), int(self.Center_x-3)) # Upper left offset point
    ]
    valid_depths = []
    for y, x in points:
        depth_val = depth_image_info[y][x]
        if depth_val != 0:
            valid_depths.append(depth_val)
    #Robot control
    if dist > 0.2: # Effective distance threshold
        self.execute(self.Center_x, dist) # Execution control logic

# Calculate the movement speed and turning angle using the PID algorithm based on
the x and y values
def execute(self, rgb_img, action):
    #PID calculation deviation
    linear_x = self.linear_pid.compute(dist, self.minDist)
    angular_z = self.angular_pid.compute(point_x,320)
    # The car is in the parking area and the speed is 0
    if abs(dist - self.minDist) < 30: linear_x = 0
    if abs(point_x - 320.0) < 30: angular_z = 0
    twist = Twist()
    ...
    # Publish the calculated speed information
    self.pub_cmdvel.publish(twist)

```

